

ECE385

DIGITAL SYSTEMS LABORATORY

Urbana Board DDR3 in RTL Designs

Abstract and Goals

The goal of this document is to assist students in creating a DDR3 (Double-Data Rate, Generation 3) based design using the Vivado MIG (Memory Interface Generator). This allows access to the 64M x 16 (1 Gigabit) DDR3 external ram chip which is located on the Urbana board. This document is designed to familiarize students with using the MIG directly via logic (through what Vivado refers to as the “User Interface” port, or UI port). First, it is important to understand the advantages and limitations of the DDR3 module, to determine if the design is appropriate for your application.

Understanding DDR3 at a High-Level

In lecture, we discussed SDR SDRAM (Single-Data Rate Synchronous Dynamic Random Access Memory). If you recall, a key limitation of most DRAMs is that the memory access time (or latency) depends on the address between the current operation and the following operation. This is due to the two-dimensional (row/column) organization of the memory. In most variants of DRAM (including both SDRAM and DDR3), memory access is fastest when accessing adjacent data words within the same row, while accesses to different rows are much slower. Therefore, most systems map the lowest bits of the address to the column address, while the higher bits of the address are assigned to the row address. This means accesses within the same row (also known as a ‘page’), can be performed with the lowest latency.

Note that when we refer to addresses, we always refer to word addresses. In the case of the Urbana board, it has a 64Mx16 RAM chip, so the addressable unit corresponds to a 16-bit word.

As each generation of DRAMs scaled to larger devices and higher frequencies to keep up with faster and more parallel CPUs and GPUs, the clock rate and bandwidth of DRAMs has increased with every generation. However, in many cases, the latency of the DRAM chips has largely stayed the same (or in some cases, even increased!) One can see this when comparing SDR SDRAM to DDR3. While the maximum random-access time for a word within an SDRAM device may be 30 ns, the same random access may take up to 150 ns with DDR3. On the other hand, an SDRAM may only transfer 200 MB/s at peak, while a comparable DDR3 device can transfer 1.3 GB/s, provided the access is to sequential words, and multiple access requests are ‘queued’ up (aka pipelined). Modern CPUs and GPUs use sophisticated caches to take full advantage of the increased bandwidth, while simultaneously hiding the effects of the increased latency of modern memory chips.

Therefore, it is important to keep in mind the capabilities and limitations of the DDR3 device and controller (MIG). DDR3 may provide very high performance for applications which require largely sequential transfers (e.g. the frame buffer we discussed in lecture). However, random accesses are extremely slow on the DDR3, comparable to the speed of a 1980s home computer (e.g. the Commodore Amiga 500 introduced in 1987 used 150 ns RAM, equivalent to

the purely random-access time of the DDR3, however the DDR3 device found on the Urbana board has over 300 times the memory **bandwidth**).

Understanding the MIG User Interface

The MIG is the Vivado provided IP which serves as the memory controller for DDR3 memory. It is implemented as a combination of hardened components (e.g. those built into the FPGA in silicon) and soft-IP components (components which are made from FPGA LUTs and registers). Because DDR3 has minimum operating frequencies which tend to be beyond the capabilities of the FPGA fabric, the MIG interface operates at either $\frac{1}{2}$ or $\frac{1}{4}$ th the frequency of the memory itself. Typically, on the Urbana board, we will use the 1:2 ratio with the memory operating at 333.3 MHz (3 ns). This will mean the MIG interface (the 'ui_clk' signal) will operate at 166.7 MHz (6 ns period).

To make up for this half-speed clock (and to account for the fact that FPGA internals usually only operate on a single clock edge, rather than transferring data on both clock edges as with DDR3), the data port going from the user design to the MIG is 4x wider than the data width of the DDR3 (e.g. the 'ui_wdf_data' and 'ui_rd_data' are both 64-bit signals for a 16-bit DDR3 device). This means that it is still possible to utilize the full memory bandwidth of the memory chip. In addition, it is important to keep in mind that the MIG is optimized for bandwidth. Therefore, it operates **only with burst reads and writes, where a burst is 8 words or 128 bits**. This can be seen by inspecting the block diagram of the memory chip. Note the bus going from the memory array to the read/write drivers is 128 bits wide. Although the addresses refer to 16-bit word addresses, accesses to the DDR3 through the MIG should always be handled in bursts of 8 words, and therefore the last 3 bits of the address given to the MIG should always be '000'. If your application requires data in a granularity of smaller than 128 bits, it is up to your application to select or mask the appropriate bits/byte/words.

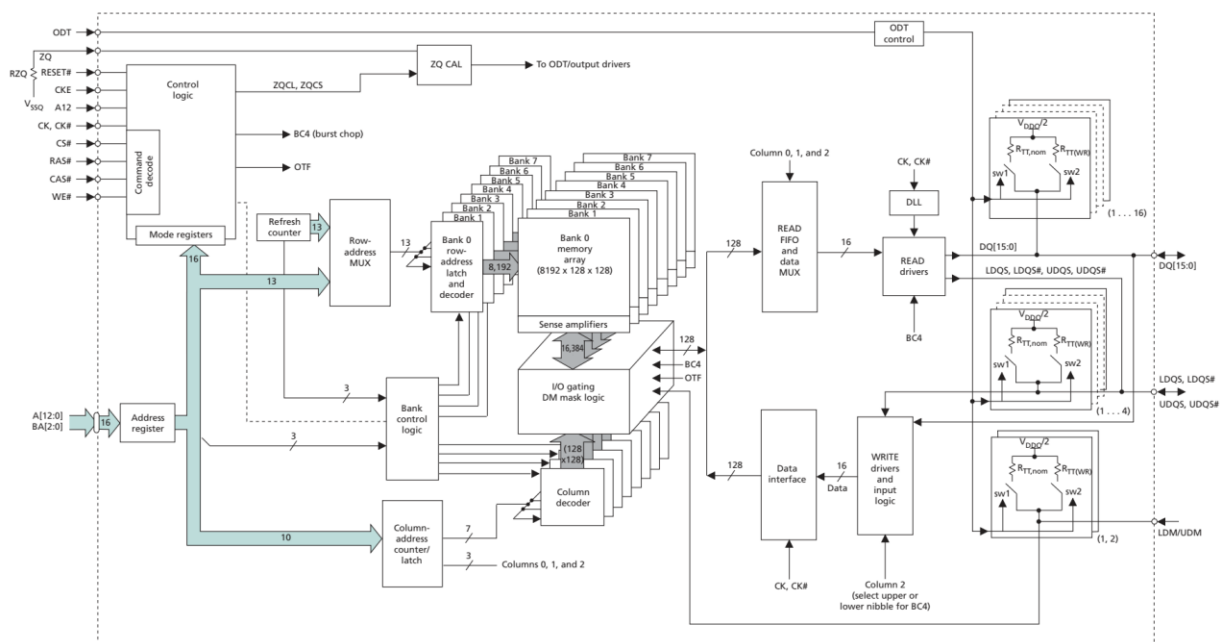


Figure 1: Micron DDR3 Block Diagram

The “User Interface” (herein abbreviated UI) which your FPGA logic uses to interface with the MIG can be thought of as 3 FIFOs. There is a command FIFO, a write data FIFO (for data FPGA → DDR3), and a read data FIFO (for data DDR3 → FPGA). The FIFOs and their relevant control signals are shown in Figure 2. What is shown is a (burst) write into address x8008. Note that the address is a multiple of 8 (the last 3 bits are ‘000’ as previously discussed). Note that there are two 64-bit words of data associated with the burst in the write data FIFO, while there is only one command associated with the same burst in the command FIFO.

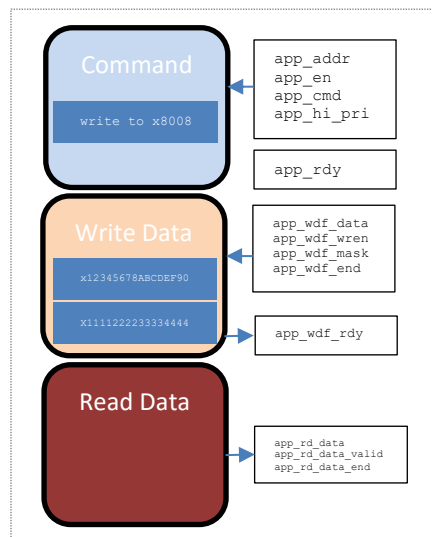


Figure 2: FIFOs of the MIG

the position of the data elements in their respective FIFOs, as that is the only way which the memory controller knows which data is associated with which address.

Note that the control signals for the command FIFO are different than the control signals for the write data FIFO, as shown in Figure 2. Vivado document UG586 specifies the exact behavior and function of each signal. For example, to load a command into the command FIFO, the following sequence in Figure 3 is observed. For a detailed explanation of each signal as shown in Figure 2, the students should consult page 74 in UG586. As a reminder when consulting the

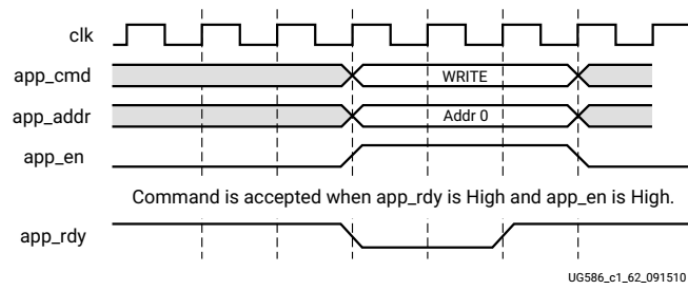


Figure 3: Loading Command and Address into Command FIFO

documentation, because we are operating in the 2:1 mode, the respect wdf_end and data_end signals will always assert on the last (or second) 64-bit data which is written or read out. In addition, you may have an application which only needs to write a single 16-bit word, rather than the entire 128-bit burst. For this purpose, the app_wdf_mask is provided, where each ‘1’ bit-position disables writing to the corresponding 16-bit word in the burst. For example, to write the second to lowest significant 16-bit word only to 0xABCD, the write FIFO should have data 0x00000000ABCD0000 for the low element and 0x0000000000000000 for the high element, while the app_wdf_mask should be 0b00000010. Also note that this takes the same amount of time as writing the full 128-bits.

Setting up the Memory Interface Generator:

Create a new blank project. Since we are going to use the HDL version of the MIG, search for “MIG” within the IP catalog.

Select next on the first customization screen for the MIG and then select “Create Design” and click next again. Notice that AXI4 Interface is unchecked, as we are using the “UI” interface.

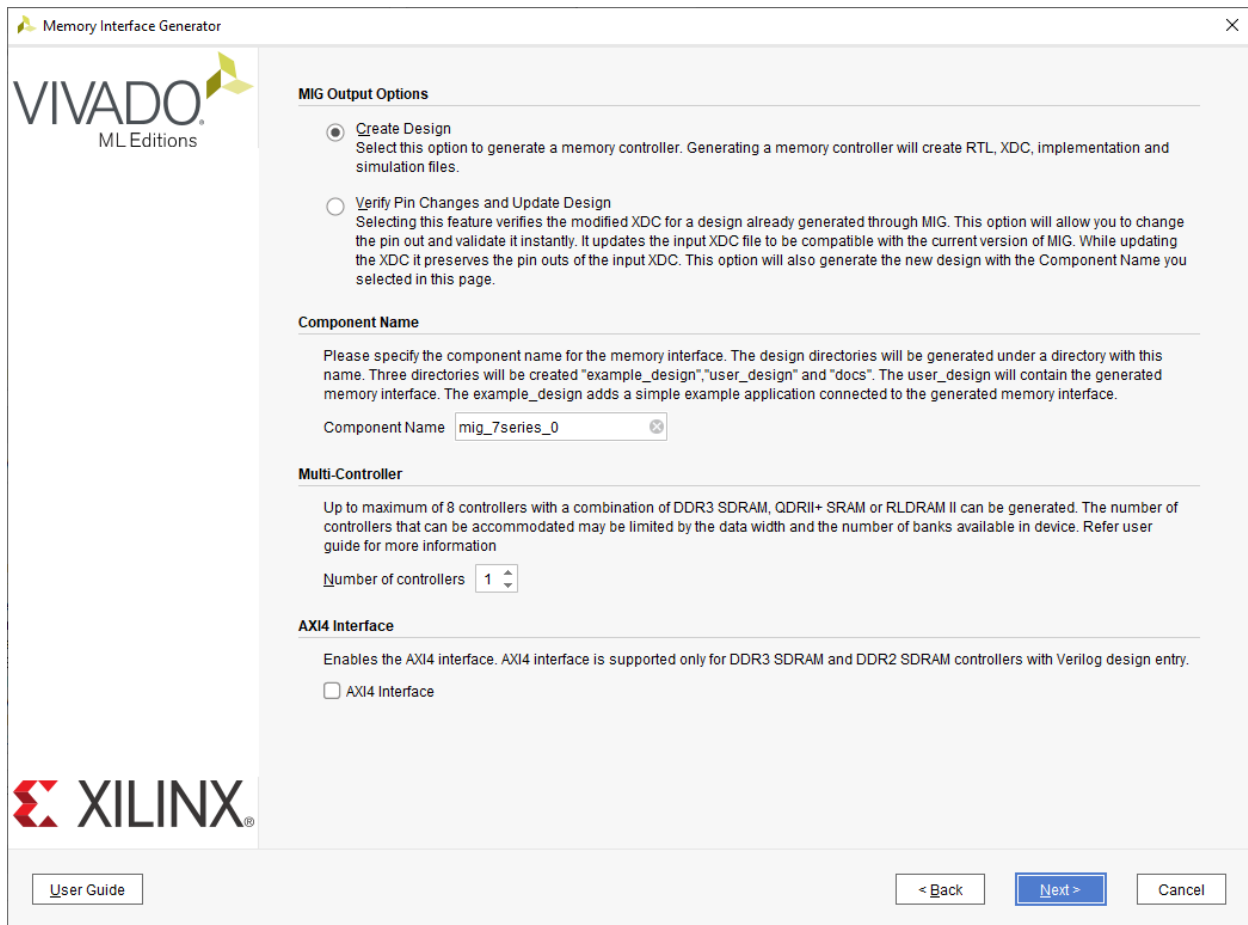


Figure 4 - MIG Intro

Click Next on the following screen (you do not need to check any of the options, as this is only used if you need to target the smaller version of the FPGA – Spartan 7 with 25K LUTs as well). Since we are only targeting a single part (Spartan 7 – 50K) we don’t need to maintain cross compatibility. On the following screen, select DDR3, and click Next. Finally, the next screen is the first where you must change a significant number of settings as follows:

Options for Controller 0 - DDR3 SDRAM

Clock Period: Choose the clock period for the desired frequency. The allowed period range(3000 - 3300) is a function of the selected FPGA part and FPGA speed grade. Refer to the User Guide for more information. ps

PHY to Controller Clock Ratio: Select the PHY to Memory Controller clock ratio. The PHY operates at the Memory Clock Period chosen above. The controller operates at either 1/4 or 1/2 of the PHY rate. The selected Memory Clock Period will limit the choices.

Memory Type: Select the memory type. Type(s) marked with a warning symbol are not compatible with the frequency selection above.

Memory Part: Select the memory part. Part(s) marked with a warning symbol are not compatible with the frequency selection above. Find an equivalent part or create a part using the "Create Custom Part" button if the part needed is not listed here. The "Create Custom Part" feature is not supported for RLDRAM II.

Memory Voltage: Select the Voltage of the Memory part selected.

Data Width: Select the Data Width. Parts marked with a warning symbol are not compatible with the frequency and memory part selected above.

ECC: MIG supports ECC for 72 bit data width configuration. To be able to select ECC, select a data width that has ECC supported.

Data Mask: Enable or disable the generation of Data Mask (DM) pins using this check box. This option can be selectable only if the memory part selected has DM pins. Uncheck this box to not use data masks and save FPGA I/Os that are used for DM signals. ECC designs (DDR3 SDRAM, DDR2 SDRAM) will not use Data Mask. ☒

Number of Bank Machines: This parameter defines the number of bank machines. A given bank machine manages a single DRAM bank at any given time.
Note: Setting a lower value will result in lower resource utilization, but may effect controller efficiency for certain traffic patterns.

ORDERING: Normal mode allows the memory controller to reorder commands to the memory to obtain the highest possible efficiency. Strict mode forces the controller to execute commands in the exact order received.

Memory Details: 1Gb, x16, row:13, col:10, bank:3, data bits per strobe:8, with data mask, single rank, 1.35V, 1.5V

Figure 5 - MIG Settings

Of particular importance is the clock period and the PHY to Controller clock ratio. The DDR3 chip you have is rated to operate between 300MHz – 400 MHz (DDR3-600 to DDR3-800), however, the memory controller on the Spartan 7 tops out at 333 MHz, which is what we will use. Since it is unlikely that the rest of your logic will run this quickly, the MIG provides a synchronous clock divider of either 4:1 or 2:1 and the UI clock will run at this speed. For now, we will use 2:1 (giving us an UI clock of 166.5 MHz). In addition, although the MIG supports up to 8 bank machines, for the most lightweight design we have reduced the # of bank machines to 2. This may reduce performance in cases where the application is reading and writing to multiple regions in the memory, however, these scenarios are rare in the simple FPGA designs for this course.

Click Next, and customize the settings as follows for the next page:

RTLDDR. 6

Memory Interface Generator

VIVADO
ML Editions

Memory Options C0 - DDR3 SDRAM

Input Clock Period: Select the period for the PLL input clock (CLKIN). MIG determines the allowable input clock periods based on the Memory Clock Period entered above and the clocking guidelines listed in the User Guide. The generated design will use the selected Input Clock and Memory Clock Periods to generate the required PLL parameters. If the required input clock period is not available, the Memory Clock Period must be modified. **9750 ps (102.564 MHz)**

Choose the Memory Options for the memory device. Memory Option selections are restricted to those supported by the controller. Consult the memory vendor data sheet for more information.

Read Burst Type and Length
The burst type determines the data ordering within a burst. Consult the memory datasheet for more information. Burst length 8 is the only supported value. **Sequential**

Output Driver Impedance Control
Programmable impedance for the output buffer. **RZQ/7**

RTT (nominal) - On Die Termination (ODT)
Select the nominal value of ODT for the DQ, DQS/DQS# and DM signals on the component or DIMM interface. This must be set to RZQ/6 ((40 ohms) for data rates at 1333 Mbps and above. In 2 slot DIMM configurations this value will be used for the unwritten slot during a write and will also be used for the unselected slot during a read. Use board level simulation to choose the optimum value. **RZQ/4**

Controller Chip Select Pin
The Chip Select (CS#) pin can be tied low externally to save one pin in the address/command group when this selection is set to 'Disable'. Disable is only valid for single rank configurations. **Disable**

Memory Address Mapping Selection

User Address

☐ ROW BANK COLUMN

☒ BANK ROW COLUMN

XILINX

[User Guide](#) [< Back](#) [Next >](#) [Cancel](#)

Figure 6 - MIG Settings 2

Of interest here is the input clock and the generation of additional clocks. Note that because 333.33 MHz (3000 ps clock) is not an even factor from 100 MHz, we must configure the MIG to 'expect' a 102.564 MHz clock instead of the actual 100 MHz oscillator. In practice, this means our design runs slightly slower than 333.33 MHz.

The screenshot shows the Vivado Memory Interface Generator (MIG) settings window. The left sidebar contains a navigation menu with options: Pin Compatible FPGAs, Memory Selection, Controller Options, AXI Parameter, Memory Options, FPGA Options, Extended FPGA Option, IO Planning Options, Pin Selection, System Signals Select, Summary, Simulation Options, PCB information, and Design Notes. The main panel displays the following settings:

- System Clock:** Choose the desired input clock configuration. Design clock can be Differential or Single-Ended. **System Clock** is set to **Differential**.
- Reference Clock:** Choose the desired reference clock configuration. Reference clock can be Differential or Single-Ended. **Reference Clock** is set to **No Buffer**.
- System Reset Polarity:** Choose the desired System Reset Polarity. **System Reset Polarity** is set to **ACTIVE HIGH**.
- Debug Signals Control:** This feature allows various debug signals present in the IP to be monitored on the ChipScope tool. The debug signals include status signals of various PHY calibration stages. Enabling this feature will connect all the debug signals to the ChipScope ILA and VIO cores in the example design top module. A part of each bus in the debug interface has been grounded so that users can replace the grounded signals with the required signals. **Debug Signals for Memory Controller** is set to **OFF**.
- Sample Data Depth:** This selects the value of Sample Data depth for Chipscope ILA used in Debug logic. **Sample Data Depth** is set to **1024**.
- Internal Vref:** Internal Vref can be used to allow the use of the Vref pins as normal IO pins. This option can only be used at 800 Mbps and lower data rates. This can free 2 pins per bank where inputs are used. This setting has no effect on banks with only outputs. **Internal Vref** is checked.
- IO Power Reduction:** Significantly reduces average IO power by automatically disabling DQ/DQS I/Os and internal terminations during WRITES and periods of inactivity. **IO Power Reduction** is set to **ON**.
- XADC Instantiation:** The memory interface uses the temperature reading from the XADC block to perform temperature compensation and keep the read DQS centered in the data window. There is one XADC block per device. If the XADC is not currently used anywhere in the design, enable this option to have the block instantiated. If the XADC is already used, disable this MIG option. The user is then required to provide the temperature value to the top level 12-bit device_temp_i input port. Refer to Answer Record 51687 or the UG586 for detailed information. **XADC Instantiation** is set to **Enabled**.

At the bottom, there are buttons for < Back, Next >, and Cancel. A User Guide button is also present in the bottom left corner.

Figure 6 - MIG Settings 3

The next page has us configure some options related to the MIG as it is generated on the FPGA. Note that unlike in previous labs, we will be using the differential clock input. This allows for a more precise clock, and it allows for the onboard clock resources on the FPGA to lock faster. Notice also that the ‘reference clock’ is chosen to be “No Buffer”. This is for an internal reference clock, which is used for calibration of the MIG. The next screen just asks for the Termination Impedance, which we will keep as the default (50 Ohm), so click Next again.

We are prompted as to whether we want to start a new design or use an existing design. We would create a new design if the Urbana Board was still in the planning phase, and we wanted to best optimize the IO utilization of the Spartan 7 FPGA. However, since the Urbana Board is already designed and realized, we should select “Fixed Pin Out”.

The next screen prompts us to load a .XDC file to verify that the pin assignments for the design are compatible with the MIG. Go ahead and load the provided .XDC files and click Validate:

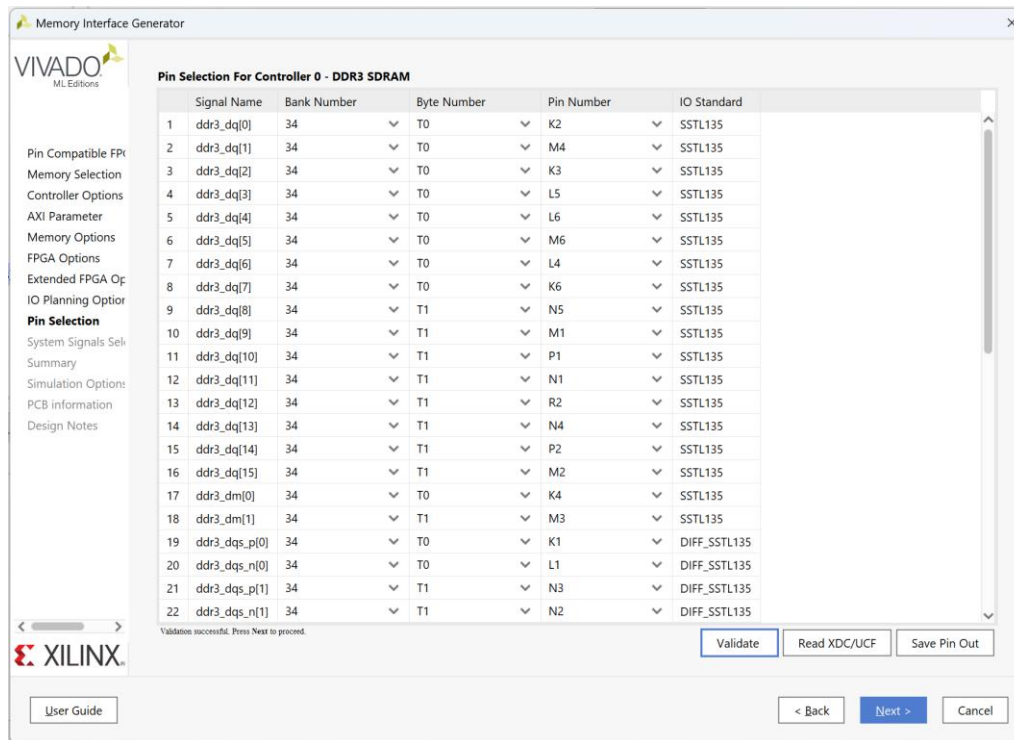


Figure 7 - XDC Validation

You should then be allowed to proceed, so click Next:

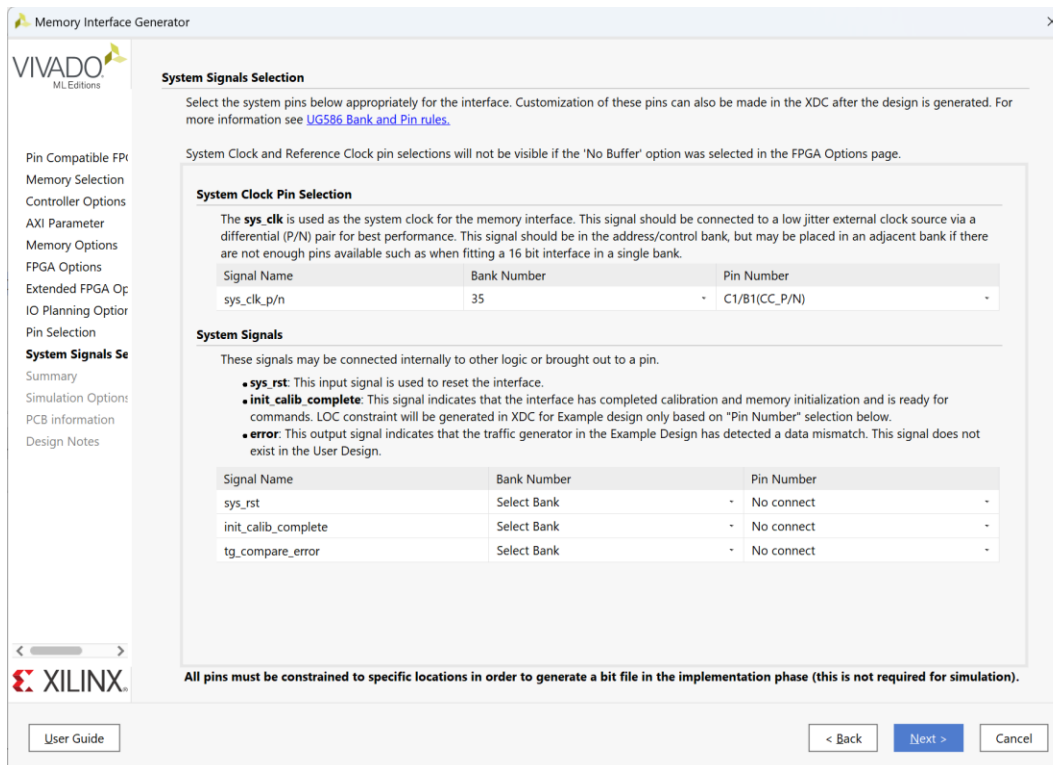


Figure 8 - Clock Selection

Now you are prompted to select the clock, which comes in on pins C1 and B1 (you may verify this with the schematic if you'd like). The other signals are optional, and therefore are not connected by default. You will then be prompted with a summary, and license agreement which you must click “Agree” and then another summary before you are prompted to click “Generate”. You will then be prompted to do an out of context run for synthesis of the MIG.

Simulating and Implementing the MIG Example Design

Now you should go back to Vivado and explore the MIG Example Design. To do this, right click on the newly created IP under sources:

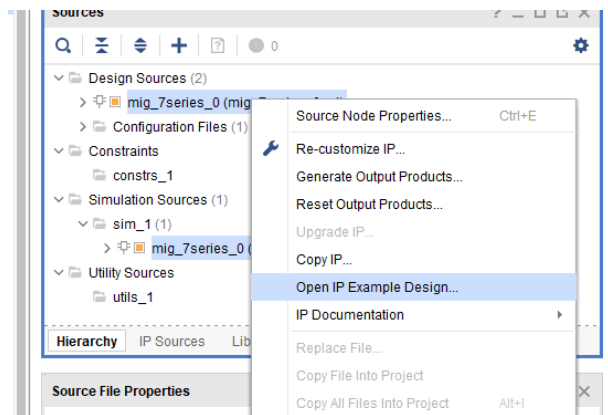


Figure 9 - IP Example Design

Enter a (short local) path for the newly created IP Example Design project. You should then be able to open the new example project. As usual, go into the Project Settings and set the Simulation Time to ‘all’, and start a (behavioral) simulation. Pay attention to both the waveform and the test printout in the TCL console.

As shown below in Figure 10, the MIG requires about 60us to perform the DDR3 training process before asserting init_calib_complete. The provided module (called the ‘traffic generator’), then implements a series of writes at specific patterns followed by reads, which the hardware TG compares. This can be observed in both simulation as well as the hardware. If the simulation runs to 50ms without detecting a read-back mismatch, the testbench will terminate after printing “TEST PASSED” in the TCL console. Note

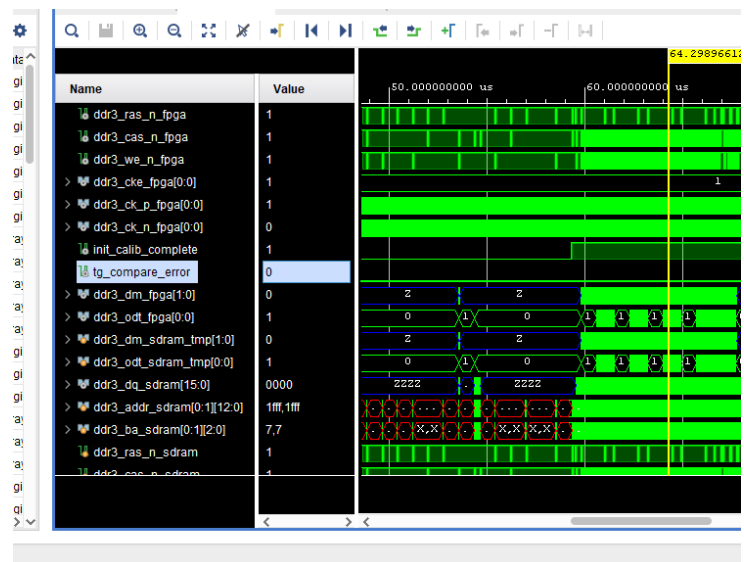


Figure 10 - Example MIG Simulation

that this may take a very long time depending on the speed of your

workstation, and so if some read/write operations appear to complete successfully, it is typically ok to cancel the simulation to inspect the read/write waveforms.

Furthermore, you can implement this design on the FPGA. In addition to the provided pin assignments for the DDR3, you should also map the following pins:

`tg_compare_error` should be mapped to a LED and indicates whether there is a memory read error (bad).

`init_calib_complete` should be mapped to a LED and indicates the DDR3 training process succeeded (good), this can also indicate to the rest of your design that the DDR3 is ready to use, and therefore may be used as a '!reset' signal for the rest of your design, so that the rest of your design stays in reset until the calibration process for DDR3 is completed.

`sys_rst` should be mapped to a pushbutton and is the overall reset for the DDR3/MIG.

At this point, once you can test the MIG in both simulation and on hardware, your goal should be to understand the user interface for the MIG. Review the section of this document called Understanding the MIG User Interface and compare that to the pre-generated traffic generator code. Use these documents to adapt the MIG UI for your own RTL based designs. If you have a question about the UI which is not answered by the document or the example design, consult chapter "Interfacing to the Core – User Interface" in Vivado User Guide document UG586.